Speech Flaw Analyzer (Track C)

Run cells top to bottom. Cells 1-14 set up the project (run once).

```
# Cell 1: install
!pip -q install librosa soundfile plotly streamlit
!pip -q install pyworld || echo 'pyworld failed - only the monotone flaw needs it'
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.3/10.3 MB 78.2
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.4/11.4 MB 107.(
       ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 261.0/261.0 kB 5.
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for pyworld (pyproject.toml) ... done
```

```
# Cell 2: create folders
!mkdir -p speechflaw scripts dataset
```

```
%%writefile speechflaw/__init__.py
"""speechflaw - contrastive speech analytics with temporal flaw grounding."""
```

```
Writing speechflaw/__init__.py
```

```
%%writefile speechflaw/config.py
"""All tunable constants live here so every run is reproducible."""

# --- signal ------------------------------------------------------------
SR = 16000                  # everything is resampled to 16 kHz mono
HOP = 160                   # 10 ms hop
FRAME = 400                 # 25 ms analysis frame
N_FFT = 512
N_MELS = 40
N_MFCC = 13
HOP_SEC = HOP / SR

# --- pitch (pYIN) ------------------------------------------------------
F0_MIN = 65.0               # Hz (C2)
F0_MAX = 400.0              # Hz (about G4) - 300 Hz was clipping real voices

# --- speech activity / pauses ------------------------------------------
ACTIVE_DB = 30.0            # frames within 30 dB of the 98th-percentile level = speech
MIN_PAUSE_SEC = 0.15        # shorter silences are articulation gaps, not pauses

# --- syllable nuclei (speech-rate proxy) -------------------------------
PEAK_PROM_DB = 4.0
PEAK_DIST = 8               # frames (80 ms)

# --- sliding-window comparison -----------------------------------------
WIN_SEC = 1.0
WIN_HOP_SEC = 0.25
MIN_ACTIVE_SEC = 0.3        # window needs this much speech to be judged
MIN_VOICED_FRAMES = 15
```

```
MIN_REGION_WINDOWS = 2     # a flaw must persist for >= 2 windows
Z_FLAG = 2.0               # flag when deviation > 2 sigma

# sigma ("tolerance band") per feature; deviation z = delta / sigma
TOL_ENERGY_DB = 2.0        # about the just-noticeable loudness difference
TOL_PITCH_ST = 1.5         # semitones of P90-P10 pitch range
TOL_RATE = 0.15            # 15 % relative change in syllable rate

# --- pauses ------------------------------------------------------------
PAUSE_MIN_FLAW_SEC = 0.7
PAUSE_FACTOR = 1.5         # x baseline 95th-percentile pause

# --- DTW ---------------------------------------------------------------
DTW_DS = 5                 # run DTW on 50 ms frames (keeps memory small)

# --- rubric weights ----------------------------------------------------
WEIGHTS = {"volume": 0.20, "pacing": 0.30, "pauses": 0.25, "pitch": 0.25}
```

```
Writing speechflaw/config.py
```

```
%%writefile speechflaw/metrics.py
"""Pure-numpy helpers (no librosa) so the logic is easy to unit-test."""
import numpy as np
from . import config as C


def runs(mask):
    """(start, end_exclusive) index pairs of consecutive True values."""
    m = np.concatenate([[False], np.asarray(mask, bool), [False]]).astype(np.int8)
    d = np.diff(m)
    return list(zip(np.where(d == 1)[0], np.where(d == -1)[0]))


def window_metrics(feat, a, b):
    """Speaker-normalised metrics over frames [a, b).

    energy : mean RMS level (dB) over speech frames
    pitch  : P90-P10 of F0 in semitones relative to the speaker's median
    rate   : syllable nuclei per second of *speech* time (pauses excluded)
    """
    nan = float("nan")
    out = {"energy": nan, "pitch": nan, "rate": nan, "speech_frac": 0.0}
    a, b = max(int(a), 0), min(int(b), feat["n"])
    if b - a < 2:
        return out
    act = feat["active"][a:b]
    n_act = int(act.sum())
    out["speech_frac"] = n_act / (b - a)
    if n_act * C.HOP_SEC < C.MIN_ACTIVE_SEC:
        return out
    out["energy"] = float(feat["rms_db"][a:b][act].mean())
    st = feat["f0_st"][a:b]
    st = st[~np.isnan(st)]
    if len(st) >= C.MIN_VOICED_FRAMES:
        out["pitch"] = float(np.percentile(st, 90) - np.percentile(st, 10))
    pk = feat["peaks"]
    n_pk = int(((pk >= a) & (pk < b)).sum())
    out["rate"] = n_pk / (n_act * C.HOP_SEC)
    return out
```

```
Writing speechflaw/metrics.py
```

```
%%writefile speechflaw/features.py
"""Acoustic feature extraction: STFT/FFT, MFCC, RMS, pYIN F0, pauses, syllable nuclei."
import numpy as np
import librosa
from scipy.ndimage import uniform_filter1d
from scipy.signal import find_peaks

from . import config as C
from .metrics import runs


def load_audio(path):
    """Load any audio file as 16 kHz mono float32."""
    y, _ = librosa.load(path, sr=C.SR, mono=True)
    return y


def extract(y):
    """Return a dict of frame-level (10 ms) features plus pauses and syllable peaks."""
    # FFT -> power spectrogram -> mel -> MFCC
    S = np.abs(librosa.stft(y, n_fft=C.N_FFT, hop_length=C.HOP, win_length=C.FRAME))
    mel = librosa.feature.melspectrogram(S=S ** 2, sr=C.SR, n_mels=C.N_MELS)
    mfcc = librosa.feature.mfcc(S=librosa.power_to_db(mel), n_mfcc=C.N_MFCC)
    centroid = librosa.feature.spectral_centroid(S=S, sr=C.SR)[0]
    flatness = librosa.feature.spectral_flatness(S=S)[0]      # noisiness / clarity pro

    # loudness (time domain RMS, in dB)
    rms = librosa.feature.rms(y=y, frame_length=C.FRAME, hop_length=C.HOP)[0]
    rms_db = 20 * np.log10(rms + 1e-6)

    # pitch (pYIN gives a proper voiced/unvoiced decision, unlike plain YIN)
    f0, vflag, _ = librosa.pyin(y, fmin=C.F0_MIN, fmax=C.F0_MAX, sr=C.SR,
                                frame_length=1024, hop_length=C.HOP)

    n = min(len(rms_db), len(f0), mfcc.shape[1], len(centroid), len(flatness))
    rms_db, f0, vflag = rms_db[:n], f0[:n], vflag[:n]
    mfcc, centroid, flatness = mfcc[:, :n], centroid[:n], flatness[:n]

    # speech activity + pauses
    thr = np.percentile(rms_db, 98) - C.ACTIVE_DB
    active = rms_db > thr
    min_p = int(round(C.MIN_PAUSE_SEC / C.HOP_SEC))
    pauses = []
    for s, e in runs(~active):
        if s == 0 or e == n:            # leading / trailing silence is not a pause
            continue
        if e - s < min_p:
            active[s:e] = True          # bridge tiny gaps
        else:
            pauses.append((s * C.HOP_SEC, e * C.HOP_SEC))

    # speaker-normalised pitch: semitones relative to this speaker's median F0
    voiced = vflag.astype(bool) & active
    f0v = np.where(voiced, f0, np.nan)
    med = float(np.nanmedian(f0v)) if voiced.sum() >= C.MIN_VOICED_FRAMES else 120.0
```

```
    f0_st = 12 * np.log2(f0v / med)

    # syllable nuclei = prominent peaks of the smoothed energy envelope
    env = uniform_filter1d(rms_db, size=5)
    peaks, _ = find_peaks(env, prominence=C.PEAK_PROM_DB, distance=C.PEAK_DIST)
    peaks = peaks[active[peaks]]

    return {
        "n": n, "duration": n * C.HOP_SEC, "rms_db": rms_db, "active": active,
        "f0_st": f0_st, "median_f0_hz": med, "mfcc": mfcc, "centroid": centroid,
        "flatness": flatness, "peaks": peaks, "pauses": pauses,
    }
```

```
Writing speechflaw/features.py
```

```
%%writefile speechflaw/align.py
"""Forced alignment: transcript words -> audio timestamps.

Primary: torchaudio MMS_FA (CTC forced aligner, needs `pip install torch torchaudio`,
downloads weights on first run). Fallback: proportional alignment over the speech span
(clearly marked method='proportional', less precise).
"""
import re
import numpy as np
from . import config as C


def normalize_words(text):
    """Lower-case letters and apostrophes only (spell numbers out in the transcript)."""
    return re.findall(r"[a-z']+", text.lower())


def _proportional(words, feat):
    idx = np.where(feat["active"])[0]
    t0 = idx[0] * C.HOP_SEC if len(idx) else 0.0
    t1 = (idx[-1] + 1) * C.HOP_SEC if len(idx) else feat["duration"]
    w = np.array([len(x) for x in words], float)
    edges = t0 + (t1 - t0) * np.concatenate([[0], np.cumsum(w) / w.sum()])
    return [{"word": x, "start": round(float(edges[i]), 3), "end": round(float(edges[i
             "score": None, "method": "proportional"} for i, x in enumerate(words)]


def forced_align(y, transcript, feat):
    words = normalize_words(transcript)
    if not words:
        return []
    try:
        import torch
        import torchaudio
        import torchaudio.functional as F
        bundle = torchaudio.pipelines.MMS_FA
        model = bundle.get_model(with_star=False).eval()
        tokenizer, aligner = bundle.get_tokenizer(), bundle.get_aligner()
        wav = torch.from_numpy(np.asarray(y, dtype=np.float32)).unsqueeze(0)
        if C.SR != bundle.sample_rate:
            wav = F.resample(wav, C.SR, bundle.sample_rate)
        with torch.inference_mode():
            emission, _ = model(wav)
```

```
        spans = aligner(emission[0], tokenizer(words))
        ratio = wav.size(1) / emission.size(1) / bundle.sample_rate
        out = []
        for w, sp in zip(words, spans):
            out.append({"word": w, "start": round(sp[0].start * ratio, 3),
                        "end": round(sp[-1].end * ratio, 3),
                        "score": round(float(np.mean([t.score for t in sp])), 3),
                        "method": "mms_fa"})
        return out
    except Exception as e:  # missing torch, no internet, odd characters...
        print(f"[align] forced aligner unavailable ({type(e).__name__}: {e}); using pro
        return _proportional(words, feat)
```

```
Writing speechflaw/align.py
```

```
%%writefile speechflaw/compare.py
"""Contrastive analysis: DTW alignment, windowed deviation, flaw regions, causal text,
import warnings
import numpy as np

from . import config as C
from .metrics import window_metrics

RULES = {  # feature -> (type when z > +Z, type when z < -Z)
    "energy": ("TOO_LOUD", "LOW_VOLUME"),
    "pitch": ("ERRATIC_PITCH", "MONOTONE"),
    "rate": ("TOO_FAST", "TOO_SLOW"),
}
TOL = {"energy": C.TOL_ENERGY_DB, "pitch": C.TOL_PITCH_ST, "rate": C.TOL_RATE}


# ---------------------------------------------------------------- DTW alignment
def dtw_map(fb, ff):
    """For every participant frame j, the matching baseline frame range [lo[j], hi[j]).
    import librosa
    ds = C.DTW_DS

    def prep(m):
        m = m[1:]                                                # drop c0 (gain dependen
        m = (m - m.mean(1, keepdims=True)) / (m.std(1, keepdims=True) + 1e-8)  # per-fi
        return m[:, ::ds]

    X, Y = prep(fb["mfcc"]), prep(ff["mfcc"])
    _, wp = librosa.sequence.dtw(X=X, Y=Y, metric="cosine")
    nX, nY = X.shape[1], Y.shape[1]
    lo = np.full(nY, nX, dtype=int)
    hi = np.zeros(nY, dtype=int)
    for i, j in wp:
        lo[j] = min(lo[j], i)
        hi[j] = max(hi[j], i)
    j_idx = np.minimum(np.arange(ff["n"]) // ds, nY - 1)
    lo_f = np.minimum(lo[j_idx] * ds, fb["n"] - 1)
    hi_f = np.minimum((hi[j_idx] + 1) * ds, fb["n"])
    return lo_f, np.maximum(hi_f, lo_f + 1)


# ---------------------------------------------------------------- helpers
def _ok(*v):
```

```
    return all(x == x for x in v)       # not NaN


def _merge(ws):
    groups, cur = [], []
    for w in ws:
        if cur and w["t0"] <= cur[-1]["t1"] + 1e-6:
            cur.append(w)
        else:
            if cur:
                groups.append(cur)
            cur = [w]
    if cur:
        groups.append(cur)
    return groups


def _nanmean(x):
    x = np.asarray(x, float)
    x = x[~np.isnan(x)]
    return float(x.mean()) if len(x) else float("nan")


def _clean(arr):
    return [None if (v is None or v != v) else round(float(v), 3) for v in arr]


def explain(r):
    t, z, bm, fm = r["type"], r["z"], r["base"], r["flaw"]
    if t in ("LOW_VOLUME", "TOO_LOUD"):
        word = "dropped" if t == "LOW_VOLUME" else "rose"
        tip = ("The passage will be hard to hear; project from the diaphragm and keep e
               if t == "LOW_VOLUME" else "Loudness overshoots the baseline; ease off so
        return (f"Loudness {word} to {fm:.1f} dB vs {bm:.1f} dB on the same words (delt
                f"sigma={C.TOL_ENERGY_DB} dB). Level is 20*log10(RMS), so amplitude is
                f"{10 ** ((fm - bm) / 20) * 100:.0f}% of the baseline. {tip}")
    if t in ("MONOTONE", "ERRATIC_PITCH"):
        tip = ("A flat F0 contour removes emphasis cues; lift pitch on key words and fa
               if t == "MONOTONE" else "Pitch swings far more than the baseline; steady
        return (f"Pitch range (P90-P10 of F0, in semitones around the speaker's own med
                f"in the baseline (delta {fm - bm:+.1f} st = {z:.1f} sigma, sigma={C.TO
    if t in ("TOO_FAST", "TOO_SLOW"):
        tip = ("Compressed syllables reduce clarity and leave no room to breathe; slow
               if t == "TOO_FAST" else "Dragging the pace loses momentum and energy; ti
        return (f"Syllable-nuclei rate is {fm:.1f}/s vs {bm:.1f}/s on the same words ({
                f"{z:.1f} sigma, sigma={C.TOL_RATE * 100:.0f}%). Rate is measured over
                f"distort it. {tip}")
    if t == "LONG_PAUSE":
        return (f"Silence of {fm:.2f}s exceeds the allowed {r['thr']:.2f}s, which is ma
                f"{C.PAUSE_FACTOR}x the baseline's 95th-percentile pause of {bm:.2f}s).
                f"flow of the argument; use short, deliberate pauses at phrase boundari
    return ""


def _region(typ, start, end, z, base, flaw, **extra):
    r = {"type": typ, "start": round(float(start), 2), "end": round(float(end), 2),
         "severity_z": round(float(z), 2), "base": float(base), "flaw": float(flaw), "z
    r.update(extra)
    r["explanation"] = explain(r)
    return r
```

```
# ------------------------------------------------------------------ main entry
def run(fb, ff, lo=None, hi=None):
    if lo is None:
        lo, hi = dtw_map(fb, ff)
    W = int(round(C.WIN_SEC / C.HOP_SEC))
    S = int(round(C.WIN_HOP_SEC / C.HOP_SEC))

    wins = []
    for a in range(0, max(1, ff["n"] - W + 1), S):
        b = min(a + W, ff["n"])
        ba, bb = int(lo[a]), int(hi[b - 1])
        if bb - ba < 5:
            continue
        mf, mb = window_metrics(ff, a, b), window_metrics(fb, ba, bb)
        z = {}
        if _ok(mf["energy"], mb["energy"]):
            z["energy"] = (mf["energy"] - mb["energy"]) / C.TOL_ENERGY_DB
        if _ok(mf["pitch"], mb["pitch"]):
            z["pitch"] = (mf["pitch"] - mb["pitch"]) / C.TOL_PITCH_ST
        if _ok(mf["rate"], mb["rate"]) and mb["rate"] > 0:
            z["rate"] = (mf["rate"] / mb["rate"] - 1) / C.TOL_RATE
        wins.append({"t0": a * C.HOP_SEC, "t1": b * C.HOP_SEC, "z": z, "f": mf, "b": mb

    regions = []
    for key, (pos, neg) in RULES.items():
        for typ, cond in ((pos, lambda v: v > C.Z_FLAG), (neg, lambda v: v < -C.Z_FLAG)
            flagged = [w for w in wins if key in w["z"] and cond(w["z"][key])]
            for g in _merge(flagged):
                if len(g) < C.MIN_REGION_WINDOWS:
                    continue
                centers = [(w["t0"] + w["t1"]) / 2 for w in g]
                start, end = max(0.0, min(centers) - 0.25), max(centers) + 0.25
                z_mean = float(np.mean([abs(w["z"][key]) for w in g]))
                regions.append(_region(typ, start, min(end, ff["duration"]), z_mean,
                                       np.mean([w["b"][key] for w in g]),
                                       np.mean([w["f"][key] for w in g]),
                                       feature=key, n_windows=len(g)))

    # pauses: compare against what the baseline speaker does naturally
    bp = [e - s for s, e in fb["pauses"]]
    ref = float(np.percentile(bp, 95)) if bp else 0.0
    thr = max(C.PAUSE_MIN_FLAW_SEC, C.PAUSE_FACTOR * ref)
    for s, e in ff["pauses"]:
        if e - s > thr:
            regions.append(_region("LONG_PAUSE", s, e, 2 * (e - s) / thr, ref, e - s,
                                   feature="pause", thr=thr))
    regions.sort(key=lambda r: r["start"])

    # rubric scores (0-100). Per window severity = clip((|z|-1)/5, 0, 1)
    def dim(key):
        s = [min(1.0, max(0.0, (abs(w["z"][key]) - 1) / 5)) for w in wins if key in w["
        return 100 * (1 - float(np.mean(s))) if s else 100.0

    pause_sev = sum(min(1.0, max(0.0, (r["z"] - 1) / 5)) for r in regions if r["type"]
    scores = {"volume": dim("energy"), "pacing": dim("rate"), "pitch": dim("pitch"),
              "pauses": 100 * (1 - min(1.0, pause_sev))}
    scores["overall"] = sum(C.WEIGHTS[k] * scores[k] for k in C.WEIGHTS)
    scores = {k: round(v, 1) for k, v in scores.items()}
```

```
    # curves for the dashboard overlay (50 ms resolution, baseline warped onto particip
    step = 5
    idx = np.arange(0, ff["n"], step)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", RuntimeWarning)
        curves = {
            "t": _clean(idx * C.HOP_SEC),
            "energy_flaw": _clean(ff["rms_db"][idx]),
            "energy_base": _clean([fb["rms_db"][lo[j]:hi[j]].mean() for j in idx]),
            "pitch_flaw": _clean([_nanmean(ff["f0_st"][j:j + step]) for j in idx]),
            "pitch_base": _clean([_nanmean(fb["f0_st"][lo[j]:hi[j]]) for j in idx]),
            "windows": [{"t": round((w["t0"] + w["t1"]) / 2, 3),
                         **{k: (round(w["z"][k], 3) if k in w["z"] else None) for k in
        }

    def art_rate(f):
        act_t = f["active"].sum() * C.HOP_SEC
        return len(f["peaks"]) / act_t if act_t > 0 else float("nan")

    glob = {
        "duration_base": round(fb["duration"], 2), "duration_flaw": round(ff["duration"
        "syllable_rate_base": round(art_rate(fb), 2), "syllable_rate_flaw": round(art_r
        "n_pauses_base": len(fb["pauses"]), "n_pauses_flaw": len(ff["pauses"]),
        "pause_threshold_sec": round(thr, 2),
        "median_f0_hz_base": round(fb["median_f0_hz"], 1), "median_f0_hz_flaw": round(f
        "mean_energy_db_base": round(float(fb["rms_db"][fb["active"]].mean()), 2),
        "mean_energy_db_flaw": round(float(ff["rms_db"][ff["active"]].mean()), 2),
        "spectral_flatness_base": round(float(fb["flatness"][fb["active"]].mean()), 4),
        "spectral_flatness_flaw": round(float(ff["flatness"][ff["active"]].mean()), 4),
    }
    return {"regions": regions, "scores": scores, "curves": curves, "global": glob}
```

```
Writing speechflaw/compare.py
```

```
%%writefile speechflaw/pipeline.py
"""End-to-end analysis + CLI:  python -m speechflaw.pipeline good.wav bad.wav --transcr
import argparse
import json

from . import align, compare, features


def analyze(base_path, flaw_path, transcript=None):
    yb, yf = features.load_audio(base_path), features.load_audio(flaw_path)
    fb, ff = features.extract(yb), features.extract(yf)
    res = compare.run(fb, ff)
    res["words"] = []
    if transcript and transcript.strip():
        words = align.forced_align(yf, transcript, ff)
        for w in words:
            w["flags"] = sorted({r["type"] for r in res["regions"]
                                 if w["end"] > r["start"] and w["start"] < r["end"]})
        res["words"] = words
        res["global"]["words_per_sec_flaw"] = round(len(words) / ff["duration"], 2)
    return res
```

```
def main():
    p = argparse.ArgumentParser()
    p.add_argument("baseline")
    p.add_argument("participant")
    p.add_argument("--transcript", help="path to a .txt transcript")
    p.add_argument("--out", default="evaluation_results.json")
    a = p.parse_args()
    text = open(a.transcript, encoding="utf-8").read() if a.transcript else None
    res = analyze(a.baseline, a.participant, text)
    with open(a.out, "w") as f:
        json.dump(res, f, indent=2)
    print(f"Overall {res['scores']['overall']}/100  |  {len(res['regions'])} flaw regio
    for r in res["regions"]:
        print(f"  {r['start']:6.2f}-{r['end']:6.2f}s  {r['type']:<14} {r['explanation']
    print(f"Saved {a.out}")


if __name__ == "__main__":
    main()
```

```
Writing speechflaw/pipeline.py
```

```
%%writefile scripts/make_dataset.py
"""Build a contrastive 'bad-mirror' gradient from one good recording of a public speech

Every flaw is injected into a known REGION, so start/end labels are exact ground truth.
Severity levels: 1 = egregious ... 5 = near-perfect.

Usage:
  python scripts/make_dataset.py --good speech.mp3 --transcript speech.txt --name obama
"""
import argparse
import csv
import pathlib
import shutil

import librosa
import numpy as np
import soundfile as sf

SR = 16000
LEVELS = {
    "gain":     [-24, -16, -10, -6, -3],              # dB change inside region
    "fast":     [2.0, 1.7, 1.45, 1.25, 1.1],          # time-stretch rate (pitch prese
    "slow":     [0.5, 0.6, 0.7, 0.8, 0.92],
    "pause":    [4.0, 2.5, 1.5, 1.0, 0.8],            # seconds of inserted silence
    "monotone": [1.0, 0.85, 0.7, 0.5, 0.3],           # fraction of pitch deviation re
}
DESC = {1: "egregious", 2: "severe", 3: "moderate", 4: "mild", 5: "near-perfect"}


def snap(y, t, search=0.3):
    """Move t to the quietest nearby frame so we never cut through a word."""
    hop = 160
    rms = librosa.feature.rms(y=y, frame_length=400, hop_length=hop)[0]
    c, r = int(t * SR / hop), int(search * SR / hop)
    lo, hi = max(c - r, 0), min(c + r, len(rms))
    return (lo + int(np.argmin(rms[lo:hi]))) * hop / SR
```

```
def apply_gain(y, a, b, db):
    env = np.ones(len(y))
    env[a:b] = 10 ** (db / 20)
    k = int(0.02 * SR)                                  # 20 ms smoothing, no clicks
    env = np.convolve(np.pad(env, (k, k), mode="edge"), np.ones(k) / k, mode="same")[k:
    return y * env, a / SR, b / SR


def apply_stretch(y, a, b, rate):
    seg = librosa.effects.time_stretch(y[a:b], rate=rate)
    return np.concatenate([y[:a], seg, y[b:]]), a / SR, a / SR + len(seg) / SR


def apply_pause(y, a, b, dur):
    return np.concatenate([y[:a], np.zeros(int(dur * SR)), y[a:]]), a / SR, a / SR + du


def apply_monotone(y, a, b, alpha):
    import pyworld as pw                                # pip install pyworld
    x = y[a:b].astype(np.float64)
    f0, t = pw.harvest(x, SR)
    sp, ap = pw.cheaptrick(x, f0, t, SR), pw.d4c(x, f0, t, SR)
    v = f0 > 0
    med = np.median(f0[v])
    f0n = f0.copy()
    f0n[v] = med * (f0[v] / med) ** (1 - alpha)
    seg = pw.synthesize(f0n, sp, ap, SR)
    seg = librosa.util.fix_length(seg, size=b - a)
    return np.concatenate([y[:a], seg.astype(y.dtype), y[b:]]), a / SR, b / SR


APPLY = {"gain": apply_gain, "fast": apply_stretch, "slow": apply_stretch,
         "pause": apply_pause, "monotone": apply_monotone}


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--good", required=True)
    p.add_argument("--transcript", required=True)
    p.add_argument("--name", required=True)
    p.add_argument("--out", default="dataset")
    p.add_argument("--region", help="start,end seconds of the region to damage (default
    p.add_argument("--region-len", type=float, default=3.0)
    a = p.parse_args()

    out = pathlib.Path(a.out)
    (out / "audio").mkdir(parents=True, exist_ok=True)
    (out / "transcripts").mkdir(exist_ok=True)

    y, _ = librosa.load(a.good, sr=SR, mono=True)
    y = (y / np.max(np.abs(y)) * 0.7).astype(np.float32)   # fixed peak level = fixed g
    dur = len(y) / SR
    if a.region:
        t0, t1 = map(float, a.region.split(","))
    else:
        t0 = snap(y, 0.3 * dur)
        t1 = snap(y, t0 + min(a.region_len, 0.35 * dur))
    ia, ib = int(t0 * SR), int(t1 * SR)
```

```
    tx = out / "transcripts" / f"{a.name}.txt"
    shutil.copy(a.transcript, tx)
    good_name = f"{a.name}_good.wav"
    sf.write(out / "audio" / good_name, y, SR)

    labels = out / "labels.csv"
    new = not labels.exists()
    fields = ["name", "role", "file", "flaw_type", "level", "level_desc", "param",
              "start_sec", "end_sec", "transcript"]
    with open(labels, "a", newline="") as f:
        w = csv.DictWriter(f, fields)
        if new:
            w.writeheader()
        w.writerow({"name": a.name, "role": "good", "file": f"audio/{good_name}", "flaw
                    "level": 0, "level_desc": "baseline", "param": "", "start_sec": "",
                    "transcript": f"transcripts/{a.name}.txt"})
        for flaw, params in LEVELS.items():
            for lvl, prm in enumerate(params, start=1):
                try:
                    yb, s, e = APPLY[flaw](y, ia, ib, prm)
                except ImportError:
                    print(f"skip {flaw}: pip install pyworld")
                    break
                fn = f"{a.name}_{flaw}_L{lvl}.wav"
                sf.write(out / "audio" / fn, np.clip(yb, -1, 1).astype(np.float32), SR)
                w.writerow({"name": a.name, "role": "bad", "file": f"audio/{fn}", "flaw
                            "level": lvl, "level_desc": DESC[lvl], "param": prm,
                            "start_sec": round(s, 3), "end_sec": round(e, 3),
                            "transcript": f"transcripts/{a.name}.txt"})
    print(f"Done. Region {t0:.2f}-{t1:.2f}s. Files in {out}/audio, labels in {labels}")


if __name__ == "__main__":
    main()
```

```
Writing scripts/make_dataset.py
```

```
%%writefile scripts/evaluate.py
"""Stress-test the detector on the dataset: detection rate, IoU and boundary error per

Usage: python scripts/evaluate.py --dataset dataset
"""
import argparse
import pathlib
import sys

import pandas as pd

sys.path.insert(0, str(pathlib.Path(__file__).resolve().parents[1]))
from speechflaw.pipeline import analyze  # noqa: E402

EXPECT = {"gain": "LOW_VOLUME", "fast": "TOO_FAST", "slow": "TOO_SLOW",
          "pause": "LONG_PAUSE", "monotone": "MONOTONE"}


def iou(a, b):
    inter = max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
```

```
    union = (a[1] - a[0]) + (b[1] - b[0]) - inter
    return inter / union if union > 0 else 0.0


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--dataset", default="dataset")
    a = p.parse_args()
    root = pathlib.Path(a.dataset)
    df = pd.read_csv(root / "labels.csv")
    rows = []
    for name, grp in df.groupby("name"):
        good = root / grp[grp.role == "good"].iloc[0]["file"]
        for _, r in grp[grp.role == "bad"].iterrows():
            tx = (root / r["transcript"]).read_text(encoding="utf-8")
            res = analyze(good, root / r["file"], tx)
            truth = (r["start_sec"], r["end_sec"])
            hits = [g for g in res["regions"] if g["type"] == EXPECT[r["flaw_type"]]]
            best = max(hits, key=lambda g: iou(truth, (g["start"], g["end"])), default=
            rows.append({
                "name": name, "flaw": r["flaw_type"], "level": r["level"],
                "detected": best is not None,
                "iou": iou(truth, (best["start"], best["end"])) if best else 0.0,
                "start_err": abs(best["start"] - truth[0]) if best else None,
                "end_err": abs(best["end"] - truth[1]) if best else None,
                "other_flags": len(res["regions"]) - len(hits),
                "overall_score": res["scores"]["overall"],
            })
            print(rows[-1])
    out = pd.DataFrame(rows)
    out.to_csv(root / "evaluation.csv", index=False)
    print("\n=== by severity level (1 egregious ... 5 near-perfect) ===")
    print(out.groupby("level")[["detected", "iou", "start_err", "end_err", "other_flags
          .mean().round(2))
    print("\n=== by flaw type ===")
    print(out.groupby("flaw")[["detected", "iou", "overall_score"]].mean().round(2))


if __name__ == "__main__":
    main()
```

```
Writing scripts/evaluate.py
```

```
%%writefile app.py
"""Interactive dashboard:  streamlit run app.py"""
import json
import os
import tempfile

import plotly.graph_objects as go
import streamlit as st
from plotly.subplots import make_subplots

from speechflaw.pipeline import analyze

st.set_page_config(page_title="Speech Flaw Analyzer", layout="wide")
COLORS = {"LOW_VOLUME": "#e74c3c", "TOO_LOUD": "#c0392b", "TOO_FAST": "#e67e22", "TOO_S
          "LONG_PAUSE": "#8e44ad", "MONOTONE": "#16a085", "ERRATIC_PITCH": "#2980b9"}
```

```
@st.cache_data(show_spinner="Extracting features, aligning and comparing...")
def run(base_bytes, base_name, flaw_bytes, flaw_name, transcript):
    paths = []
    for data, name in ((base_bytes, base_name), (flaw_bytes, flaw_name)):
        f = tempfile.NamedTemporaryFile(delete=False, suffix=os.path.splitext(name)[1]
        f.write(data)
        f.close()
        paths.append(f.name)
    try:
        return analyze(paths[0], paths[1], transcript)
    finally:
        for p in paths:
            os.unlink(p)


st.title("Contrastive Speech Analytics")
st.caption("Upload a good baseline and a participant recording of the same speech. "
           "Speaker-agnostic: pitch is in semitones around each speaker's median.")

with st.sidebar:
    base = st.file_uploader("Baseline (ideal) audio", type=["wav", "mp3", "flac", "m4a"
    flaw = st.file_uploader("Participant audio", type=["wav", "mp3", "flac", "m4a", "og
    tx_file = st.file_uploader("Transcript (.txt, optional)", type=["txt"])
    go_btn = st.button("Analyze", type="primary", disabled=not (base and flaw))

if not (base and flaw):
    st.info("Upload both audio files in the sidebar to begin.")
    st.stop()

st.audio(base.getvalue(), format=base.type or "audio/wav")
st.caption("Baseline")
st.audio(flaw.getvalue(), format=flaw.type or "audio/wav")
st.caption("Participant")

if not go_btn and "res" not in st.session_state:
    st.stop()
if go_btn:
    tx = tx_file.getvalue().decode("utf-8") if tx_file else None
    st.session_state["res"] = run(base.getvalue(), base.name, flaw.getvalue(), flaw.nam
res = st.session_state["res"]

# ---- scores
sc = res["scores"]
cols = st.columns(5)
cols[0].metric("Overall", f"{sc['overall']:.0f}/100")
for c, k in zip(cols[1:], ["volume", "pacing", "pauses", "pitch"]):
    c.metric(k.title(), f"{sc[k]:.0f}")

# ---- overlay
cv = res["curves"]
fig = make_subplots(rows=3, cols=1, shared_xaxes=True, vertical_spacing=0.07,
                    subplot_titles=("Loudness (dB)", "Pitch (semitones vs speaker media
                                    "Deviation from baseline (sigma units, flag at +/-2
t = cv["t"]
fig.add_trace(go.Scatter(x=t, y=cv["energy_base"], name="Baseline (time-warped)", line=
fig.add_trace(go.Scatter(x=t, y=cv["energy_flaw"], name="Participant", line=dict(color=
fig.add_trace(go.Scatter(x=t, y=cv["pitch_base"], name="Baseline pitch", line=dict(colo
                         showlegend=False, connectgaps=False), 2, 1)
```

```
    fig.add_trace(go.Scatter(x=t, y=cv["pitch_flaw"], name="Participant pitch", line=dict(c
                             showlegend=False, connectgaps=False), 2, 1)
    wt = [w["t"] for w in cv["windows"]]
    for key, color in (("energy", "#e74c3c"), ("pitch", "#16a085"), ("rate", "#e67e22")):
        fig.add_trace(go.Scatter(x=wt, y=[w[key] for w in cv["windows"]], name=f"z {key}",
                                 line=dict(color=color)), 3, 1)
    for y0 in (2, -2):
        fig.add_hline(y=y0, line_dash="dot", line_color="gray", row=3, col=1)
    for r in res["regions"]:
        for row in (1, 2, 3):
            fig.add_vrect(x0=r["start"], x1=r["end"], row=row, col=1, fillcolor=COLORS.get(
                          opacity=0.18, line_width=0, layer="below")
    fig.update_xaxes(title_text="Participant time (s)", row=3, col=1)
    fig.update_layout(height=760, margin=dict(t=60, b=20), legend=dict(orientation="h", y=1
    st.plotly_chart(fig, use_container_width=True)

    # ---- flaw regions + causal explanations
    st.subheader("Flaw regions")
    if not res["regions"]:
        st.success("No region deviates from the baseline by more than 2 sigma.")
    for r in res["regions"]:
        with st.expander(f"{r['start']:.2f}s - {r['end']:.2f}s  |  {r['type']}  |  {r['seve
            st.write(r["explanation"])

    # ---- transcript with highlighted words
    if res["words"]:
        st.subheader("Transcript")
        html = []
        for w in res["words"]:
            if w["flags"]:
                c = COLORS.get(w["flags"][0], "#999")
                html.append(f"<mark style='background:{c}55' title='{w['start']}s: {', '.jo
            else:
                html.append(w["word"])
        st.markdown(" ".join(html), unsafe_allow_html=True)
        st.caption(f"Alignment method: {res['words'][0]['method']}")

    with st.expander("Raw feature summary"):
        st.json(res["global"])
    st.download_button("Download results (JSON)", json.dumps(res, indent=2), "evaluation_re
```

```
Writing app.py
```

```
%%writefile requirements.txt
numpy
scipy
pandas
librosa>=0.10
soundfile
plotly
streamlit
torch
torchaudio
pyworld
```

```
Writing requirements.txt
```

Quick test: upload ANY zip (or loose audio files) of the same speech

```
import sys, glob, zipfile, os, shutil, textwrap
sys.path.insert(0, '.')
from google.colab import files
from speechflaw.pipeline import analyze

BASELINE = None   # optional: part of the baseline file name, e.g. 'good'
EXT = ('.wav', '.mp3', '.flac', '.m4a', '.ogg')

up = files.upload()
shutil.rmtree('audio_data', ignore_errors=True)
os.makedirs('audio_data')
for name in up:
    if name.lower().endswith('.zip'):
        zipfile.ZipFile(name).extractall('audio_data')
    elif name.lower().endswith(EXT):
        shutil.copy(name, 'audio_data')

audio = sorted(p for p in glob.glob('audio_data/**/*', recursive=True)
               if p.lower().endswith(EXT) and '__MACOSX' not in p
               and not os.path.basename(p).startswith('._'))
print(f"Found {len(audio)} audio files:")
for p in audio:
    print("  ", p)

keys = [BASELINE] if BASELINE else ['normal', 'good', 'baseline', 'ideal', 'original',
base = next((p for k in keys for p in audio if k.lower() in os.path.basename(p).lower()
print("\nBASELINE:", base)

def report(name, r):
    s, g = r['scores'], r['global']
    dims = {k: s[k] for k in ('volume', 'pacing', 'pauses', 'pitch')}
    worst = min(dims, key=dims.get)
    print('\n' + '=' * 70)
    print(os.path.basename(name))
    print(f"OVERALL {s['overall']}/100 | volume {s['volume']} | pacing {s['pacing']} |
    print(f"Weakest area: {worst} ({dims[worst]}/100)")
    print(f"Duration {g['duration_base']}s -> {g['duration_flaw']}s | syllables/s {g['s
          f"loudness {g['mean_energy_db_base']} -> {g['mean_energy_db_flaw']} dB | paus
    if not r['regions']:
        print("\nNo region deviates from the baseline by more than 2 sigma, so delivery
    for i, reg in enumerate(r['regions'], 1):
        print(f"\n[{i}] {reg['start']:.2f}s to {reg['end']:.2f}s | {reg['type']} | {reg
        print(textwrap.fill("WHY: " + reg['explanation'], 90, subsequent_indent='     '

results = {}
for w in audio:
    if w == base:
        continue
    results[w] = analyze(base, w)
    report(w, results[w])
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving speech_audio_files.zip to speech_audio_files.zip
Found 4 audio files:
   audio_data/speech_01_normal .mp3
   audio_data/speech_02_pause .mp3
   audio_data/speech_03_low_mumbling_volume-1.mp3
   audio_data/speech_04_extremly rushed,rushed .mp3

BASELINE: audio_data/speech_01_normal .mp3

======================================================================
speech_02_pause .mp3
OVERALL 79.0/100 | volume 94.5 | pacing 75.9 | pauses 75.6 | pitch 73.8
Weakest area: pitch (73.8/100)
Duration 21.71s -> 33.23s | syllables/s 3.78 -> 2.91 | loudness -26.7 -> -27.43 dB | pau

[1] 0.25s to 1.25s | TOO_SLOW | 3.83 sigma
WHY: Syllable-nuclei rate is 1.7/s vs 3.9/s on the same words (-57%, 3.8 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[2] 1.25s to 3.00s | ERRATIC_PITCH | 2.55 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 9.5 st
     vs 5.7 st in the baseline (delta +3.8 st = 2.6 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[3] 1.75s to 2.50s | LOW_VOLUME | 2.67 sigma
WHY: Loudness dropped to -27.3 dB vs -22.0 dB on the same words (delta -5.3 dB = 2.7
     sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 54% of the
     baseline. The passage will be hard to hear; project from the diaphragm and keep
     energy up to the end of phrases.

[4] 3.44s to 4.35s | LONG_PAUSE | 2.22 sigma
WHY: Silence of 0.91s exceeds the allowed 0.82s, which is max(0.7s, 1.5x the baseline's
     95th-percentile pause of 0.55s). Long unplanned silence breaks the flow of the
     argument; use short, deliberate pauses at phrase boundaries instead.

[5] 6.00s to 6.75s | TOO_SLOW | 5.48 sigma
WHY: Syllable-nuclei rate is 0.9/s vs 3.8/s on the same words (-76%, 5.5 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[6] 7.50s to 9.00s | MONOTONE | 2.68 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 7.1 st
     vs 11.2 st in the baseline (delta -4.0 st = 2.7 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[7] 8.00s to 9.50s | TOO_SLOW | 2.96 sigma
WHY: Syllable-nuclei rate is 3.1/s vs 5.4/s on the same words (-43%, 3.0 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[8] 9.00s to 12.50s | ERRATIC_PITCH | 3.57 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 10.4 s
     vs 5.1 st in the baseline (delta +5.4 st = 3.6 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[9] 9.50s to 10.50s | TOO_FAST | 2.48 sigma
WHY: Syllable-nuclei rate is 3.6/s vs 2.6/s on the same words (+37%, 2.5 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[10] 10.50s to 13.75s | TOO_SLOW | 3.73 sigma
WHY: Syllable-nuclei rate is 1.9/s vs 4.2/s on the same words (-56%, 3.7 sigma,
```

WHY: Syllable-nuclei rate is 1.9/s vs 4.2/s on the same words (-56%, 3.7 sigma,
sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
Dragging the pace loses momentum and energy; tighten the delivery.

[11] 12.50s to 13.75s | LOW_VOLUME | 2.65 sigma
WHY: Loudness dropped to -30.4 dB vs -25.1 dB on the same words (delta -5.3 dB = 2.6
sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 54% of the
baseline. The passage will be hard to hear; project from the diaphragm and keep
energy up to the end of phrases.

[12] 14.75s to 16.25s | TOO_SLOW | 4.33 sigma
WHY: Syllable-nuclei rate is 1.5/s vs 3.8/s on the same words (-60%, 4.3 sigma,
sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
Dragging the pace loses momentum and energy; tighten the delivery.

[13] 15.00s to 15.75s | LOW_VOLUME | 3.1 sigma
WHY: Loudness dropped to -31.5 dB vs -25.3 dB on the same words (delta -6.2 dB = 3.1
sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 49% of the
baseline. The passage will be hard to hear; project from the diaphragm and keep
energy up to the end of phrases.

[14] 15.00s to 16.00s | MONOTONE | 4.52 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 2.8 st
vs 9.6 st in the baseline (delta -6.8 st = 4.5 sigma, sigma=1.5 st). A flat F0
contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[15] 15.75s to 16.75s | ERRATIC_PITCH | 3.4 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 7.9 st
vs 2.8 st in the baseline (delta +5.1 st = 3.4 sigma, sigma=1.5 st). Pitch swings f
more than the baseline; steady the melody so it sounds controlled.

[16] 16.75s to 17.50s | TOO_FAST | 4.58 sigma
WHY: Syllable-nuclei rate is 3.1/s vs 1.8/s on the same words (+70%, 4.6 sigma,
sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
Compressed syllables reduce clarity and leave no room to breathe; slow down on key
phrases.

[17] 18.25s to 19.00s | TOO_LOUD | 2.32 sigma
WHY: Loudness rose to -25.1 dB vs -29.7 dB on the same words (delta +4.6 dB = 2.3 sigma,
sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 171% of the baseline.
Loudness overshoots the baseline; ease off so emphasis still stands out.

[18] 19.75s to 20.50s | ERRATIC_PITCH | 6.03 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 15.1 s
vs 6.0 st in the baseline (delta +9.1 st = 6.0 sigma, sigma=1.5 st). Pitch swings f
more than the baseline; steady the melody so it sounds controlled.

[19] 23.00s to 23.75s | ERRATIC_PITCH | 3.53 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 10.2 s
vs 4.9 st in the baseline (delta +5.3 st = 3.5 sigma, sigma=1.5 st). Pitch swings f
more than the baseline; steady the melody so it sounds controlled.

[20] 25.25s to 26.75s | TOO_SLOW | 3.92 sigma
WHY: Syllable-nuclei rate is 2.0/s vs 4.8/s on the same words (-59%, 3.9 sigma,
sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
Dragging the pace loses momentum and energy; tighten the delivery.

[21] 26.25s to 28.75s | MONOTONE | 5.05 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 8.7 st
vs 16.3 st in the baseline (delta -7.6 st = 5.1 sigma, sigma=1.5 st). A flat F0
contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[22] 29.00s to 29.75s | ERRATIC_PITCH | 2.26 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 10.4 s
vs 7.0 st in the baseline (delta +3.4 st = 2.3 sigma, sigma=1.5 st). Pitch swings f
more than the baseline; steady the melody so it sounds controlled.

```
[23] 29.00s to 30.00s | TOO_SLOW | 3.44 sigma
WHY: Syllable-nuclei rate is 2.8/s vs 5.8/s on the same words (-52%, 3.4 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[24] 29.50s to 30.75s | MONOTONE | 5.52 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 4.3 st
     vs 12.6 st in the baseline (delta -8.3 st = 5.5 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[25] 31.00s to 31.75s | ERRATIC_PITCH | 2.88 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 11.0 s
     vs 6.7 st in the baseline (delta +4.3 st = 2.9 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[26] 31.50s to 32.50s | MONOTONE | 5.48 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 10.7 s
     vs 18.9 st in the baseline (delta -8.2 st = 5.5 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

======================================================================
speech_03_low_mumbling_volume-1.mp3
OVERALL 82.7/100 | volume 76.9 | pacing 77.0 | pauses 100 | pitch 76.9
Weakest area: volume (76.9/100)
Duration 21.71s -> 30.36s | syllables/s 3.78 -> 3.46 | loudness -26.7 -> -29.77 dB | pau

[1] 0.25s to 5.25s | LOW_VOLUME | 5.35 sigma
WHY: Loudness dropped to -35.3 dB vs -24.6 dB on the same words (delta -10.7 dB = 5.4
     sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 29% of the
     baseline. The passage will be hard to hear; project from the diaphragm and keep
     energy up to the end of phrases.

[2] 0.25s to 2.75s | TOO_FAST | 6.54 sigma
WHY: Syllable-nuclei rate is 5.6/s vs 2.9/s on the same words (+93%, 6.5 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[3] 5.00s to 6.50s | MONOTONE | 2.42 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 8.4 st
     vs 12.0 st in the baseline (delta -3.6 st = 2.4 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[4] 6.00s to 6.75s | TOO_SLOW | 3.0 sigma
WHY: Syllable-nuclei rate is 3.0/s vs 5.6/s on the same words (-47%, 3.0 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[5] 7.50s to 10.25s | LOW_VOLUME | 4.42 sigma
WHY: Loudness dropped to -35.8 dB vs -27.0 dB on the same words (delta -8.8 dB = 4.4
     sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 36% of the
     baseline. The passage will be hard to hear; project from the diaphragm and keep
     energy up to the end of phrases.

[6] 7.75s to 9.00s | TOO_FAST | 6.27 sigma
WHY: Syllable-nuclei rate is 4.4/s vs 2.5/s on the same words (+74%, 6.3 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[7] 9.75s to 10.50s | TOO_FAST | 4.2 sigma
WHY: Syllable-nuclei rate is 4.0/s vs 2.5/s on the same words (+64%, 4.2 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.
```

```
[8] 10.75s to 12.00s | ERRATIC_PITCH | 2.91 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 13.3 s
     vs 9.0 st in the baseline (delta +4.4 st = 2.9 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[9] 11.00s to 13.25s | TOO_SLOW | 3.0 sigma
WHY: Syllable-nuclei rate is 2.4/s vs 4.3/s on the same words (-44%, 3.0 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[10] 12.75s to 14.25s | LOW_VOLUME | 3.1 sigma
WHY: Loudness dropped to -30.9 dB vs -24.7 dB on the same words (delta -6.2 dB = 3.1
     sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 49% of the
     baseline. The passage will be hard to hear; project from the diaphragm and keep
     energy up to the end of phrases.

[11] 12.75s to 14.00s | MONOTONE | 4.62 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 2.6 st
     vs 9.6 st in the baseline (delta -6.9 st = 4.6 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[12] 13.75s to 15.00s | ERRATIC_PITCH | 3.13 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 6.0 st
     vs 1.3 st in the baseline (delta +4.7 st = 3.1 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[13] 15.25s to 16.50s | TOO_FAST | 4.2 sigma
WHY: Syllable-nuclei rate is 2.0/s vs 1.3/s on the same words (+59%, 4.2 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[14] 16.50s to 17.50s | TOO_SLOW | 3.23 sigma
WHY: Syllable-nuclei rate is 1.9/s vs 3.5/s on the same words (-47%, 3.2 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[15] 24.25s to 25.00s | ERRATIC_PITCH | 2.66 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 13.0 s
     vs 9.0 st in the baseline (delta +4.0 st = 2.7 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[16] 24.25s to 25.25s | TOO_SLOW | 2.73 sigma
WHY: Syllable-nuclei rate is 3.9/s vs 6.7/s on the same words (-41%, 2.7 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[17] 24.75s to 26.00s | LOW_VOLUME | 2.37 sigma
WHY: Loudness dropped to -30.2 dB vs -25.5 dB on the same words (delta -4.7 dB = 2.4
     sigma, sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 58% of the
     baseline. The passage will be hard to hear; project from the diaphragm and keep
     energy up to the end of phrases.

[18] 28.50s to 29.75s | MONOTONE | 6.11 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 8.6 st
     vs 17.8 st in the baseline (delta -9.2 st = 6.1 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[19] 28.50s to 29.75s | TOO_SLOW | 2.85 sigma
WHY: Syllable-nuclei rate is 2.2/s vs 3.8/s on the same words (-42%, 2.8 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

======================================================================
speech_04_extremly rushed,rushed .mp3
```

```
OVERALL 84.7/100 | volume 84.7 | pacing 86.4 | pauses 100 | pitch 67.6
Weakest area: pitch (67.6/100)
Duration 21.71s -> 17.72s | syllables/s 3.78 -> 4.01 | loudness -26.7 -> -23.9 dB | paus

[1] 0.50s to 1.25s | TOO_SLOW | 2.15 sigma
WHY: Syllable-nuclei rate is 2.5/s vs 3.7/s on the same words (-32%, 2.1 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Dragging the pace loses momentum and energy; tighten the delivery.

[2] 1.25s to 4.25s | MONOTONE | 5.55 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 6.3 st
     vs 14.6 st in the baseline (delta -8.3 st = 5.5 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[3] 1.50s to 3.75s | TOO_FAST | 3.77 sigma
WHY: Syllable-nuclei rate is 5.5/s vs 3.6/s on the same words (+52%, 3.8 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[4] 2.50s to 3.50s | TOO_LOUD | 2.73 sigma
WHY: Loudness rose to -22.6 dB vs -28.0 dB on the same words (delta +5.5 dB = 2.7 sigma,
     sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 187% of the baseline.
     Loudness overshoots the baseline; ease off so emphasis still stands out.

[5] 4.25s to 6.75s | TOO_LOUD | 2.23 sigma
WHY: Loudness rose to -21.0 dB vs -25.4 dB on the same words (delta +4.5 dB = 2.2 sigma,
     sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 167% of the baseline.
     Loudness overshoots the baseline; ease off so emphasis still stands out.

[6] 5.50s to 7.50s | MONOTONE | 2.92 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 8.2 st
     vs 12.6 st in the baseline (delta -4.4 st = 2.9 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

[7] 7.75s to 9.25s | TOO_LOUD | 2.85 sigma
WHY: Loudness rose to -21.9 dB vs -27.6 dB on the same words (delta +5.7 dB = 2.9 sigma,
     sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 193% of the baseline.
     Loudness overshoots the baseline; ease off so emphasis still stands out.

[8] 9.00s to 11.00s | MONOTONE | 5.33 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 9.1 st
     vs 17.1 st in the baseline (delta -8.0 st = 5.3 sigma, sigma=1.5 st). A flat F0
     contour removes emphasis cues; lift pitch on key words and fall at sentence ends.

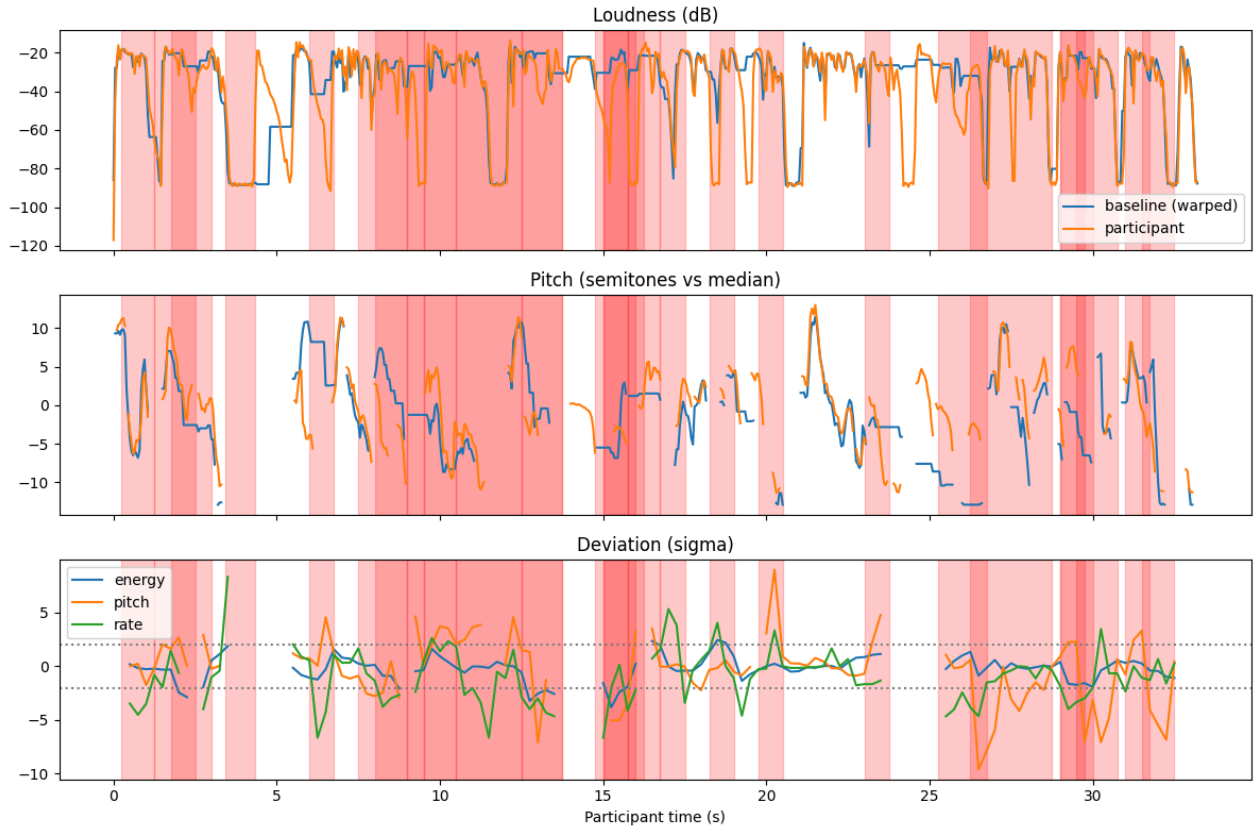
[9] 9.00s to 9.75s | TOO_FAST | 3.33 sigma
WHY: Syllable-nuclei rate is 4.0/s vs 2.7/s on the same words (+49%, 3.3 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.

[10] 10.50s to 12.25s | TOO_LOUD | 2.9 sigma
WHY: Loudness rose to -21.8 dB vs -27.6 dB on the same words (delta +5.8 dB = 2.9 sigma,
     sigma=2.0 dB). Level is 20*log10(RMS), so amplitude is about 195% of the baseline.
     Loudness overshoots the baseline; ease off so emphasis still stands out.

[11] 10.75s to 12.00s | ERRATIC_PITCH | 2.84 sigma
WHY: Pitch range (P90-P10 of F0, in semitones around the speaker's own median) is 11.2 s
     vs 6.9 st in the baseline (delta +4.3 st = 2.8 sigma, sigma=1.5 st). Pitch swings f
     more than the baseline; steady the melody so it sounds controlled.

[12] 12.00s to 13.25s | TOO_FAST | 2.94 sigma
WHY: Syllable-nuclei rate is 6.9/s vs 4.8/s on the same words (+44%, 2.9 sigma,
     sigma=15%). Rate is measured over speech time only, so pauses do not distort it.
     Compressed syllables reduce clarity and leave no room to breathe; slow down on key
     phrases.
```

```
import matplotlib.pyplot as plt
w = list(results)[0]
r = results[w]; cv = r['curves']
fig, ax = plt.subplots(3, 1, figsize=(12, 8), sharex=True)
ax[0].plot(cv['t'], cv['energy_base'], label='baseline (warped)'); ax[0].plot(cv['t'],
ax[1].plot(cv['t'], cv['pitch_base'], label='baseline'); ax[1].plot(cv['t'], cv['pitch_
for k in ('energy', 'pitch', 'rate'):
    ax[2].plot([x['t'] for x in cv['windows']], [x[k] if x[k] is not None else float('n
ax[2].axhline(2, ls=':', c='gray'); ax[2].axhline(-2, ls=':', c='gray'); ax[2].set_titl
for g in r['regions']:
    for a in ax:
        a.axvspan(g['start'], g['end'], alpha=0.2, color='red')
plt.xlabel('Participant time (s)'); plt.tight_layout(); plt.show()
```



Build the dataset from a real speech

```
from google.colab import files
up = files.upload()
print(list(up.keys()))
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving speech_01_normal .mp3 to speech_01_normal .mp3
['speech_01_normal .mp3']
```

```
!pip -q install transformers
import os
from transformers import pipeline
```

```
GOOD = list(up.keys())[0]
new_name = 'speech' + os.path.splitext(GOOD)[1].strip()
os.rename(GOOD, new_name)
GOOD = new_name
NAME = 'speech1'

asr = pipeline("automatic-speech-recognition", model="openai/whisper-small", chunk_leng
TEXT = asr(GOOD, generate_kwargs={"language": "english"})["text"].strip()
print("TRANSCRIPT:", TEXT)
open('speech.txt', 'w').write(TEXT)

!python scripts/make_dataset.py --good {GOOD} --transcript speech.txt --name {NAME} --o
!head -5 dataset/labels.csv
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to
config.json: 100%  1.97k/1.97k [00:00<00:00, 77.7kB/s]
model.safetensors: reconstructing file: 100%  967MB / 967MB, 54.2MB/s
model.safetensors: downloading bytes:  643MB, 46.0MB/s
Loading weights: 100%  479/479 [00:00<00:00, 2227.38it/s]
generation_config.json: 100%  3.87k/3.87k [00:00<00:00, 56.5kB/s]
tokenizer_config.json: 100%  283k/283k [00:00<00:00, 4.55MB/s]
vocab.json: 100%  836k/836k [00:00<00:00, 7.21MB/s]
tokenizer.json: 100%  2.48M/2.48M [00:00<00:00, 23.1MB/s]
merges.txt: 100%  494k/494k [00:00<00:00, 7.32MB/s]
normalizer.json: 100%  52.7k/52.7k [00:00<00:00, 792kB/s]
added_tokens.json: 100%  34.6k/34.6k [00:00<00:00, 585kB/s]
special_tokens_map.json: 100%  2.19k/2.19k [00:00<00:00, 27.5kB/s]
preprocessor_config.json: 100%  185k/185k [00:00<00:00, 4.19MB/s]
[transformers] Using `chunk_length_s` is very experimental with seq2seq models. The resu
[transformers] Passing `generation_config` together with generation-related arguments=({
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer WhisperToken
TRANSCRIPT: Good morning, everyone. My name is Rohit. Today, I want to talk about the im
/usr/local/lib/python3.13/dist-packages/pyworld/__init__.py:13: UserWarning: pkg_resourc
  import pkg_resources
Done. Region 6.30-9.57s. Files in dataset/audio, labels in dataset/labels.csv
name,role,file,flaw_type,level,level_desc,param,start_sec,end_sec,transcript
speech1,good,audio/speech1_good.wav,none,0,baseline,,,,transcripts/speech1.txt
speech1,bad,audio/speech1_gain_L1.wav,gain,1,egregious,-24,6.3,9.57,transcripts/speech1.
speech1,bad,audio/speech1_gain_L2.wav,gain,2,severe,-16,6.3,9.57,transcripts/speech1.txt
speech1,bad,audio/speech1_gain_L3.wav,gain,3,moderate,-10,6.3,9.57,transcripts/speech1.t
```

```
!python scripts/evaluate.py --dataset dataset
```

```
Downloading: "https://dl.fbaipublicfiles.com/mms/torchaudio/ctc_alignment_mling_uroman/m
100% 1.18G/1.18G [00:11<00:00, 108MB/s]
{'name': 'speech1', 'flaw': 'gain', 'level': 1, 'detected': True, 'iou': 0.8175000000000
{'name': 'speech1', 'flaw': 'gain', 'level': 2, 'detected': True, 'iou': 0.8720000000000
{'name': 'speech1', 'flaw': 'gain', 'level': 3, 'detected': True, 'iou': 0.8963585434173
{'name': 'speech1', 'flaw': 'gain', 'level': 4, 'detected': True, 'iou': 0.9174311926605
```

```
{'name': 'speech1', 'flaw': 'gain', 'level': 5, 'detected': False, 'iou': 0.0, 'start_er
{'name': 'speech1', 'flaw': 'fast', 'level': 1, 'detected': True, 'iou': 0.2325581395348
{'name': 'speech1', 'flaw': 'fast', 'level': 2, 'detected': False, 'iou': 0.0, 'start_er
{'name': 'speech1', 'flaw': 'fast', 'level': 3, 'detected': False, 'iou': 0.0, 'start_er
{'name': 'speech1', 'flaw': 'fast', 'level': 4, 'detected': False, 'iou': 0.0, 'start_er
{'name': 'speech1', 'flaw': 'fast', 'level': 5, 'detected': True, 'iou': 0.0, 'start_err
{'name': 'speech1', 'flaw': 'slow', 'level': 1, 'detected': True, 'iou': 0.6116207951070
{'name': 'speech1', 'flaw': 'slow', 'level': 2, 'detected': True, 'iou': 0.6422018348623
{'name': 'speech1', 'flaw': 'slow', 'level': 3, 'detected': True, 'iou': 0.6422607578676
{'name': 'speech1', 'flaw': 'slow', 'level': 4, 'detected': True, 'iou': 0.3058478101296
{'name': 'speech1', 'flaw': 'slow', 'level': 5, 'detected': False, 'iou': 0.0, 'start_er
{'name': 'speech1', 'flaw': 'pause', 'level': 1, 'detected': True, 'iou': 0.9975, 'start
{'name': 'speech1', 'flaw': 'pause', 'level': 2, 'detected': True, 'iou': 0.996000000000
{'name': 'speech1', 'flaw': 'pause', 'level': 3, 'detected': True, 'iou': 0.993333333333
{'name': 'speech1', 'flaw': 'pause', 'level': 4, 'detected': True, 'iou': 0.990000000000
{'name': 'speech1', 'flaw': 'pause', 'level': 5, 'detected': False, 'iou': 0.0, 'start_e
{'name': 'speech1', 'flaw': 'monotone', 'level': 1, 'detected': True, 'iou': 0.688073394
{'name': 'speech1', 'flaw': 'monotone', 'level': 2, 'detected': True, 'iou': 0.688073394
{'name': 'speech1', 'flaw': 'monotone', 'level': 3, 'detected': True, 'iou': 0.688073394
{'name': 'speech1', 'flaw': 'monotone', 'level': 4, 'detected': True, 'iou': 0.688073394
{'name': 'speech1', 'flaw': 'monotone', 'level': 5, 'detected': True, 'iou': 0.535168195

=== by severity level (1 egregious ... 5 near-perfect) ===
       detected   iou  start_err  end_err  other_flags  overall_score
```